

JLA_V2编解码库开发说明

声明

- 本项目所参考、使用技术必须全部来源于公知技术信息，或自主创新设计。
- 本项目不得使用任何未经授权的第三方知识产权的技术信息。
- 如个人使用未经授权的第三方知识产权的技术信息，造成的经济损失和法律后果由个人承担

历史版本

一、概述

二、接口说明

- 2.1 JLA_V2编解码接口
- 2.2 编解码参数
- 2.3 状态事件回调器
- 2.4 流式状态事件回调器
- 2.5 常量定义
- 2.6 错误码

三、使用说明

- 3.1 解码文件
- 3.2 解码数据流
- 3.3 编码文件
- 3.4 编码数据流

历史版本

版本	日期	更新内容	负责人
1.0.0	2025/5/30	1. 增加 JLA_V2编解码 说明	钟卓成

一、概述

本文档是为了方便用户接入JLA_V2编解码库而创建，用户可以通过该文档快速接入JLA_V2编解码功能。

JLA_V2编解码库已支持功能：

- JLA_V2解码文件
- JLA_V2解码数据流
- JLA_V2编码文件
- JLA_V2编码数据流

二、接口说明

2.1 JLA_V2编解码接口

JLAudioV2Codec

```
public class JLAudioV2Codec {

    public JLAudioV2Codec() throws RuntimeException;

    /**
     * 解码JLA_V2文件
     *
     * @param inFilePath String 输入JLA_V2编码文件路径
     * @param outFilePath String 输出Pcm文件路径
     * @param param CodecParam 编解码参数
     * @param callback OnStateCallback 解码状态回调
     */
    public void decodeFile(String inFilePath, String outFilePath, CodecParam
param, OnStateCallback callback);

    /**
     * 是否正在解码
     *
     * @return boolean 结果
     */
    public boolean isDecoding();

    /**
     * 开始解码JLA_V2数据流
     *
     * @param param CodecParam 编解码参数
     * @param callback OnStreamStateCallback 流式解码状态回调
     */
    public void startDecodeStream(CodecParam param, OnStreamStateCallback
callback);

    /**
     * 停止解码JLA_V2数据流
     *
     * @return boolean 结果
     * <p>若结果为true，停止结果将在 {@link OnStreamStateCallback#onStop(String)} 回调</p>
     */
    public boolean stopDecodeStream();

    /**
     * 写入待解码的JLA_V2数据流
     *
     * @param data byte[] JLA_V2数据流
     */
}
```

```

        * @return boolean 结果
        */
    public boolean writeDecodeData(byte[] data);

    /**
     * 编码JLA_V2文件
     *
     * @param inFilePath String 输入PCM文件路径
     * @param outFilePath String 输出JLA_V2文件路径
     * @param param CodecParam 编解码参数
     * @param callback OnStateCallback 解码状态回调
     */
    public void encodeFile(String inFilePath, String outFilePath, CodecParam
param, OnStateCallback callback);

    /**
     * 是否正在编码
     *
     * @return boolean 结果
     */
    public boolean isEncoding();

    /**
     * 开始编码PCM数据流
     *
     * @param param CodecParam 编解码参数
     * @param callback OnStreamStateCallback 流式解码状态回调
     */
    public void startEncodeStream(CodecParam param, OnStreamStateCallback
callback);

    /**
     * 停止编码PCM数据流
     *
     * @return boolean 结果
     * <p>
     * 若结果为true，停止结果将在 {@link OnStreamStateCallback#onStop(String)} 回调
     * </p>
     */
    public boolean stopEncodeStream();

    /**
     * 写入待编码的PCM数据
     *
     * @param data byte[] PCM数据
     * @return boolean 结果
     */
    public boolean writeEncodeData(byte[] data);

    /**
     * 释放编解码器
     */
    public void release();
}

```

2.2 编解码参数

CodecParam

```
public class CodecParam implements Parcelable {

    /**
     * 帧长数组
     */
    public static final int[] FRAME_LEN_ARRAY = new int[]{32, 40, 48, 60, 64, 80,
    96, 120, 128, 160, 240, 320, 400, 480};

    /**
     * 采样率
     * <p>单位: Hz, 必须项</p>
     */
    private long sampleRate = 16000;

    /**
     * 码率
     * <p>
     * 单位:bps, 必须项
     * </p>
     */
    private long bitRate = 48000;

    /**
     * 帧长索引
     * <p>
     * 必须项
     * </p>
     */
    private int frameLenIdx = 9;

    /**
     * 声道数
     * <p>
     * 必须项
     * </p>
     */
    private int channel = 1;

    /**
     * 输入输出数据是否为24Bit对齐数据
     * <p>
     * 必须项
     * </p>
     */
    private boolean iss24 = false;
}
```

2.3 状态事件回调器

OnStateCallback

```
public interface OnStateCallback {  
    /**  
     * 回调操作开始  
     */  
    void onStart();  
  
    /**  
     * 回调操作完成  
     *  
     * @param message String 信息  
     */  
    void onStop(String message);  
  
    /**  
     * 回调操作失败  
     *  
     * @param code int 错误码  
     * @param message String 错误描述  
     */  
    void onError(int code, String message);  
}
```

2.4 流式状态事件回调器

OnStreamStateCallback

```
public interface OnStreamStateCallback extends OnStateCallback {  
  
    /**  
     * 回调流式数据  
     *  
     * @param data byte[] 数据  
     */  
    void onStreamData(byte[] data);  
}
```

2.5 常量定义

Constants

```
public class Constants {  
  
    /** ===== *  
     * 操作类型
```

```

    * ===== */

/**
 * 解码文件类型
 */
public static final int TYPE_DECODE_FILE = 1;
/**
 * 编码文件类型
 */
public static final int TYPE_ENCODE_FILE = 2;
/**
 * 流式解码类型
 */
public static final int TYPE_DECODE_STREAM = 3;
/**
 * 流式编码类型
 */
public static final int TYPE_ENCODE_STREAM = 4;

/* ===== *
 * 状态码
 * ===== */

/**
 * 空闲状态
 */
public static final int STATE_IDLE = 0;
/**
 * 工作状态
 */
public static final int STATE_WORKING = 1;
}

```

2.6 错误码

ErrorCode

```

public class ErrorCode {
    /**
     * 操作成功
     */
    public static final int ERR_NONE = 0;
    /**
     * 未初始化
     */
    public static final int ERR_NOT_INIT = 1;
    /**
     * 参数错误
     */
    public static final int ERR_INVALID_PARAM = 2;
}

```

```

/**
 * 操作中
 */
public static final int ERR_IN_PROGRESS = 3;
/**
 * IO异常
 */
public static final int ERR_IO_EXCEPTION = 4;
/**
 * 打开文件失败
 */
public static final int ERR_OPEN_FILE = 5;
/**
 * 分配空间失败
 */
public static final int ERR_MALLOC_FAIL = 6;
/**
 * 错误数据
 */
public static final int ERR_WRONG_DATA = 7;
/**
 * SDK已销毁
 */
public static final int ERR_SDK_RELEASE = 128;
}

```

三、使用说明

3.1 解码文件

```

String inFilePath = "JLA_V2文件路径";
String outFilePath = "PCM输出文件路径";
try {
    JLAudioV2Codec codec = new JLAudioV2Codec();
    if (codec.isDecoding()) return; //正在解码中
    CodecParam param = new CodecParam();
    param.setSampleRate(16000) //设置采样率为 16K
        .setSampleRate(16000) //设置码率为 16K
        .setFrameLenIdx(11); //设置帧长为 320 Bytes
    //执行解码文件接口
    codec.decodeFile(inFilePath, outFilePath, param, new onStateCallback() {
        @Override
        public void onStart() {
            //回调解码开始
        }

        @Override
        public void onStop(String message) {
            //回调解码结束
            //message --- 输出文件路径
        }
    });
}

```

```

    }

    @Override
    public void onError(int code, String message) {
        //回调解码出错
        //code --- 错误码
        //message --- 错误描述
    }
};
//销毁编解码器
// codec.release();
} catch (RuntimeException e) {
    e.printStackTrace(); //初始化失败
}

```

说明

1. [CodecParam](#)
2. [OnStateCallback](#)
3. [错误码](#)

3.2 解码数据流

```

try {
    JLAudioV2Codec codec = new JLAudioV2Codec();
    if (codec.isDecoding()) return; //正在解码中
    CodecParam param = new CodecParam();
    param.setSampleRate(16000) //设置采样率为 16K
        .setSampleRate(16000) //设置码率为 16K
        .setFrameLenIdx(11); //设置帧长为 320 Bytes
    //执行解码流数据接口
    codec.startDecodeStream(param, new OnStreamStateCallback() {
        @Override
        public void onStreamData(byte[] data) {
            //回调解码数据
        }

        @Override
        public void onStart() {
            //回调解码开始
        }

        @Override
        public void onStop(String message) {
            //回调解码结束
            //message --- 结束信息
        }

        @Override
        public void onError(int code, String message) {
            //回调解码出错
        }
    });
}

```



```

        //code --- 错误码
        //message --- 错误描述
    }
});
//添加解码数据
byte[] jlav2Data = new byte[0]; //需要解码的JLA_V2数据
codec.writeDecodeData(jlav2Data);
//停止流式解码
codec.stopDecodeStream();
//销毁编解码器
//    codec.release();
} catch (RuntimeException e) {
    e.printStackTrace(); //初始化失败
}

```

说明

1. [CodecParam](#)
2. [OnStreamStateCallback](#)
3. [错误码](#)

3.3 编码文件

```

String inFilePath = "PCM文件路径";
String outFilePath = "JLA_V2文件路径";
try {
    JLAudioV2Codec codec = new JLAudioV2Codec();
    if (codec.isEncoding()) return; //正在编码中
    CodecParam param = new CodecParam();
    param.setSampleRate(16000) //设置采样率为 16K
        .setSampleRate(16000) //设置码率为 16K
        .setFrameLenIdx(11); //设置帧长为 320 Bytes
    //执行编码文件接口
    codec.encodeFile(inFilePath, outFilePath, param, new OnStateCallback() {
        @Override
        public void onStart() {
            //回调编码开始
        }

        @Override
        public void onStop(String message) {
            //回调编码结束
            //message --- 输出文件路径
        }

        @Override
        public void onError(int code, String message) {
            //回调编码出错
            //code --- 错误码
            //message --- 错误描述
        }
    });
}

```

```

    });
    //销毁编解码器
    // codec.release();
} catch (RuntimeException e) {
    e.printStackTrace(); //初始化失败
}

```

说明

1. [CodecParam](#)
2. [OnStateCallback](#)
3. [错误码](#)

3.4 编码数据流

```

try {
    JLAudioV2Codec codec = new JLAudioV2Codec();
    if (codec.isEncoding()) return; //正在编码中
    CodecParam param = new CodecParam();
    param.setSampleRate(16000) //设置采样率为 16k
        .setSampleRate(16000) //设置码率为 16k
        .setFrameLenIdx(11); //设置帧长为 320 Bytes
    //执行解码流数据接口
    codec.startEncodeStream(param, new OnStreamStateCallback() {
        @Override
        public void onStreamData(byte[] data) {
            //回调编码数据
        }

        @Override
        public void onStart() {
            //回调编码开始
        }

        @Override
        public void onStop(String message) {
            //回调编码结束
            //message --- 结束信息
        }

        @Override
        public void onError(int code, String message) {
            //回调编码出错
            //code --- 错误码
            //message --- 错误描述
        }
    });
    //添加编码数据
    byte[] pcmData = new byte[0]; //需要编码的PCM数据
    codec.writeEncodeData(pcmData);
    //停止流式编码
}

```

```
        codec.stopEncodeStream();  
        //销毁编解码器  
        //    codec.release();  
    } catch (RuntimeException e) {  
        e.printStackTrace(); //初始化失败  
    }
```

说明

1. [CodecParam](#)
2. [OnStreamStateCallback](#)
3. [错误码](#)